

# 2차 피드백에 대한 수정사항

(해당 페이지 **하이라이트**)

(우하단 페이지번호 기준)

| 페이지 번호 | 피드백                                | 피드백 반영 사항                           |
|--------|------------------------------------|-------------------------------------|
| 13     | PPT 가독성 향상 ("맞춤형 경량화 달성."의 마침표 제거) | 수정 완료                               |
| 11, 19 | 프로젝트 목표 scope 줄이기                  | 메모리 절감 아이디어 5건에서 3건으로 하향 조정         |
| 23, 24 | 역할분담 및 향후 일정 세부화 및 최신화             | 해당사항 간단하게 추가 완료                     |
| -      | - (중간발표 05/01 (금) 까지, )            | 향후 중간발표까지 ppt구조 개편, 가독성 향상 등 수정할 예정 |



# Chromium

## 메모리 기능 관리

## 대시보드 개발

**SNU X LG Electronics**

2026.03.20

Spec Presentation

H조 노현제, 손민규, 한진우

# Overview

---

프로젝트 배경 및 개요



# Chromium이란?

---

Linux Kernel과 함께 세계 최대 규모의 오픈소스 프로젝트

166,223

연간 Commits

Chrome, Edge, Brave, Whale, Samsung Internet 등  
주요 브라우저가 모두 크로미움 기반으로 개발

2,649

활성 Contributors

LG webOS, Samsung Tizen 등  
임베디드 시스템 플랫폼에서도 핵심 엔진으로 활용

C++ 54%

주요 언어

방대한 최적화 환경:  
수백 개의 커스텀 빌드 옵션 및 런타임 플래그 제공

4주

업그레이드 주기

# 프로젝트 배경

---

## 01 webOS와 Chromium

2013년 LG는 HP로부터 webOS 인수 이후,  
TV·자동차·로봇 등 다양한 임베디드 디바이스에 Chromium 기반 웹 엔진 탑재

## 02 메모리 경량화의 경제적 가치

메모리가 1-2GB인 임베디드 환경 특성상, 단 10MB의 경량화도 원가 절감으로 직결  
최근 1년 새 D램 가격 약 10배 급등 (3.9만 → 37.4만원)

## 03 기능 유지보수의 한계

메모리 경량화 기능을 이미 다수 확보했으나  
4주 단위의 잦은 업데이트 주기로 인해 수동 검증은 사실상 불가능

webOS

# Problems & Goals

---

문제 정의 및 프로젝트 목표

# 문제점 및 목표

---

## Problems

### 짧은 릴리즈 주기, 수동 검증의 비효율

4주 단위 크로미움 업데이트에 대응하여,  
수많은 메모리 기능을 빌드·실행·측정 과정의 수동 반복 불가

### 데이터 파편화

측정 결과의 체계적 저장 및 분석 도구 부재

### 인사이트 도출 한계

기능 간 시너지 및 Trade-off (메모리 vs 성능) 파악의 어려움

## Goals

### 01 자동화 검증 파이프라인 구축

빌드·실행·측정 전 과정 자동화  
동일 수행 시 동일 결과 보장 (재현성)

### 02 메모리 기능 관리 대시보드 개발

버전별 누적 데이터 시각화  
기능 조합별 시너지 분석

### 03 메모리 절감 아이디어 5건+ 발굴

커맨드라인 스위치 ~ 코드 수정 수준까지

# Approach

---

문제 접근 방식

# 1. 자동 검증 시스템 개발

---

## 01 단일 스크립트 제어

단일 파이썬 스크립트로 빌드·실행·측정 전 과정 통제  
런타임 플래그, 빌드 옵션, 코드 패치 등 적용 방식에 따른 자동 분기 처리

## 02 원격 Headless 실행

물리 모니터가 없는 Ubuntu 서버 환경  
원격 디버깅 포트 오픈 및 연동

## 03 CDP 기반 정밀 측정

Selenium을 활용한 CDP 제어  
시나리오: 지정 URL 20초 대기 이후 20초간 측정 (MAX 값 도출)

- **Target:** 유튜브, 네이버스포츠 등 제공된 URL 3종
- **Metric:** JSHeapUsedSize 등 지표

## 04 결과 데이터 구조화

동일 과정 5회 반복 수행 후 중간값 채택  
여러 데이터를 JSON 파일로 구조화하여 대시보드와 연동

---

※ CDP: Chrome DevTools Protocol, 크로미움 브라우저를 외부에서 원격으로 제어·측정할 수 있는 공식 API

※ JSHeapUsedSize: CDP의 Runtime.getHeapUsage 함수가 반환하는 자바스크립트의 메모리 사용량

# 1. 자동 검증 시스템 개발

---

## 극복해야 할 기술적 과제

### 01 원격 Headless 제어

물리적 디스플레이가 없는 서버에서 브라우저 실행 제약  
Headless 모드 빌드 및 xvfb 가상 디스플레이 시뮬레이션 병행

### 02 측정 재현성 보장

GC, 캐시 등 백그라운드 노이즈로 인한 측정 오차 발생  
노이즈 통제 및 다중 측정으로 신뢰성 있는 대표값 추출

### 03 빌드 및 스토리지 최적화

40GB+ 방대한 소스코드와 서버의 제한된 디스크 용량  
빌드 산출물 삭제 로직을 파이프라인에 포함하여 스토리지 최적화

### 04 멀티프로세스 지표 선별

탭마다 별도 프로세스로 동작하는 크로미움의 메모리 구조  
공유·비공유 메모리를 구분하고, CDP를 활용해 적합한 지표만 선별

※ xvfb: 실제 모니터 없이 GUI 화면을 가상으로 구성해주는 리눅스 도구

※ 공유 메모리: GPU 버퍼 등 여러 프로세스가 동시에 접근하는 메모리 영역. 메모리 사용량 측정 시 중복 계산되지 않도록 처리가 필요함

## 2. 메모리 기능 관리 대시보드 개발

---

### 01

#### 기능별 절감량 시각화

단일 기능 별 메모리 절감 효과를  
그래프로 직관적 시각화

각 기능의 기술적 요약 제공

### 02

#### 버전별 추이 관리

크로미움 버전 단위 누적 데이터로  
이전 대비 증감폭 한눈에 파악

4주 주기 업그레이드 대응

### 03

#### 동적 기능 조합 분석

대시보드 내에서 동적으로  
새로운 조합 구성 및 비교

개별 합산 대비 시너지 및  
Trade-off 표시

---

### 구현 방향

- Grafana 등 무거운 기존 프레임워크 대신 Flask, Vanilla JS, Chart.js 기반 자체 경량 웹 개발
- 파이프라인 JSON 결과물 실시간 연동 및 대시보드 내 파이프라인 직접 제어가 가능한 인터랙티브 UI 구축

# 3. 추가 메모리 절감 아이디어 구상

---

## Level 1

### 커맨드라인 스위치

실행 시 --flag 옵션으로 메모리 절감 기능 ON/OFF

--single-process (보안 ↓ 중복 라이브러리 메모리 ↓)

optimize\_for\_size=true (속도 ↓ 코드 크기 ↓)

--disable-features=Bluetooth (불필요 API 제거, ~10MB)

## Level 2

### 코드/빌드 수정

크로미움 소스코드 직접 수정 또는 빌드 설정(GN args) 변경

Cobalt 엔진 기여 사례 참고

불필요 웹 표준 API 컴포넌트 제거

목표: 다양한 유형(플래그, 빌드, 패치)을 포함한 **신규 절감 기능 3건+ 발굴**

검증: 구축된 자동화 파이프라인을 통해 실제 절감 효과 테스트 및 교차 검증

# 3. 추가 메모리 절감 아이디어 구상

Mojo: 크로미움의 비동기 메시지 패싱(통신) 프레임워크

IPC: 프로세스 간 통신 (Inter-Process Communication)

## 실제 사례 분석: Cobalt 엔진

### PR #6439 컴포지터(Compositor) 캐시 및 타일 매니저 메모리 축소

로직: 화면 렌더링을 담당하는 컴포지터 계층의 이미지 캐시 및 타일 매니저 메모리 상한선 하향 조정

효과: 렌더링 파이프라인의 잉여 캐시 용량을 제한하여 수십 MB 규모의 그래픽 메모리 절감 기대

### PR #6644 8K 비디오 버퍼 예산(Budget) 최적화

로직: 8K 초고해상도 비디오 재생 시 미디어 버퍼(Video Buffer Budget) 할당량 세분화 및 하향 조정

효과: 8K 스트리밍 시 수백 MB 단위로 팽창할 수 있는 미디어 버퍼 오버헤드를 선제적으로 차단하여 메모리 고갈 방지

### PR #9145 Mojo IPC 일괄 처리를 통한 동적 할당 오버헤드 절감

로직: 미디어 파이프라인의 샘플 데이터를 하나의 연속된 메모리 블록에 묶어 일괄 처리

효과: 잦은 통신으로 객체당 발생하는 수 KB~MB 수준의 미세 오버헤드를 일괄 처리하여 누적 메모리 사용량 대폭 감소

# 3. 추가 메모리 절감 아이디어 구상

---

## 실제 사례 분석: Cobalt 엔진, V8 Lite Mode

### 01 도메인 특화 경량화 (Cobalt 엔진 사례)

배경: 범용 브라우저인 크로미움은 방대한 웹 표준 API를 탑재하고 있으나, 스마트 TV 등 임베디드 기기에서는 사용되지 않는 기능이 다수 존재

적용: 타겟 환경(webOS 등)의 특성을 고려하여 화상 회의(WebRTC)나 블루투스 제어 등 불필요한 모듈을 빌드 단계에서 통째로 제거

효과: 기능 제한을 통해 수십 MB 이상의 메모리를 선제적으로 확보하며 **맞춤형 경량화 달성**

### 02 Trade-off 기반의 공식 옵션화 (V8 Lite Mode 사례)

배경: 자바스크립트 최적화 컴파일 과정은 실행 속도를 높이지만 상당한 메모리 공간을 요구함

적용: 엔진 최적화 기능을 일부 제한하여 실행 속도(Performance)를 소폭 양보하는 대신, 메모리 사용량을 극적으로 절감하는 모드 개발

의의: 저사양 기기를 위한 특수 패치에서 시작해, 그 효용성을 인정받아 현재 공식 빌드 옵션(v8\_enable\_lite\_mode)으로 편입된 대표적 사례

# 3. 추가 메모리 절감 아이디어 구상

---

## 기타 사례

### 03 보안 정책 완화 및 프로세스 통합

단일 프로세스(--single-process) 구동으로 중복 라이브러리 메모리 대폭 절감

### 04 백그라운드 탭 메모리 강제 반환

비활성 상태의 앱 상태만 임시 저장하고 메모리를 OS에 강제 회수

### 05 GPU 래스터라이제이션 및 텍스처 제한

하드웨어 가속 제한 및 해상도 하향을 통한 공유 메모리 확보 (Trade-off)

### 06 가비지 컬렉션(GC) 스케줄링 최적화

유휴 시간(Idle time)에만 GC를 동작시켜 평상시 메모리 노이즈 억제

# Specifications

---

프로젝트 세부 요구사항

# 1. 자동 검증 시스템

---

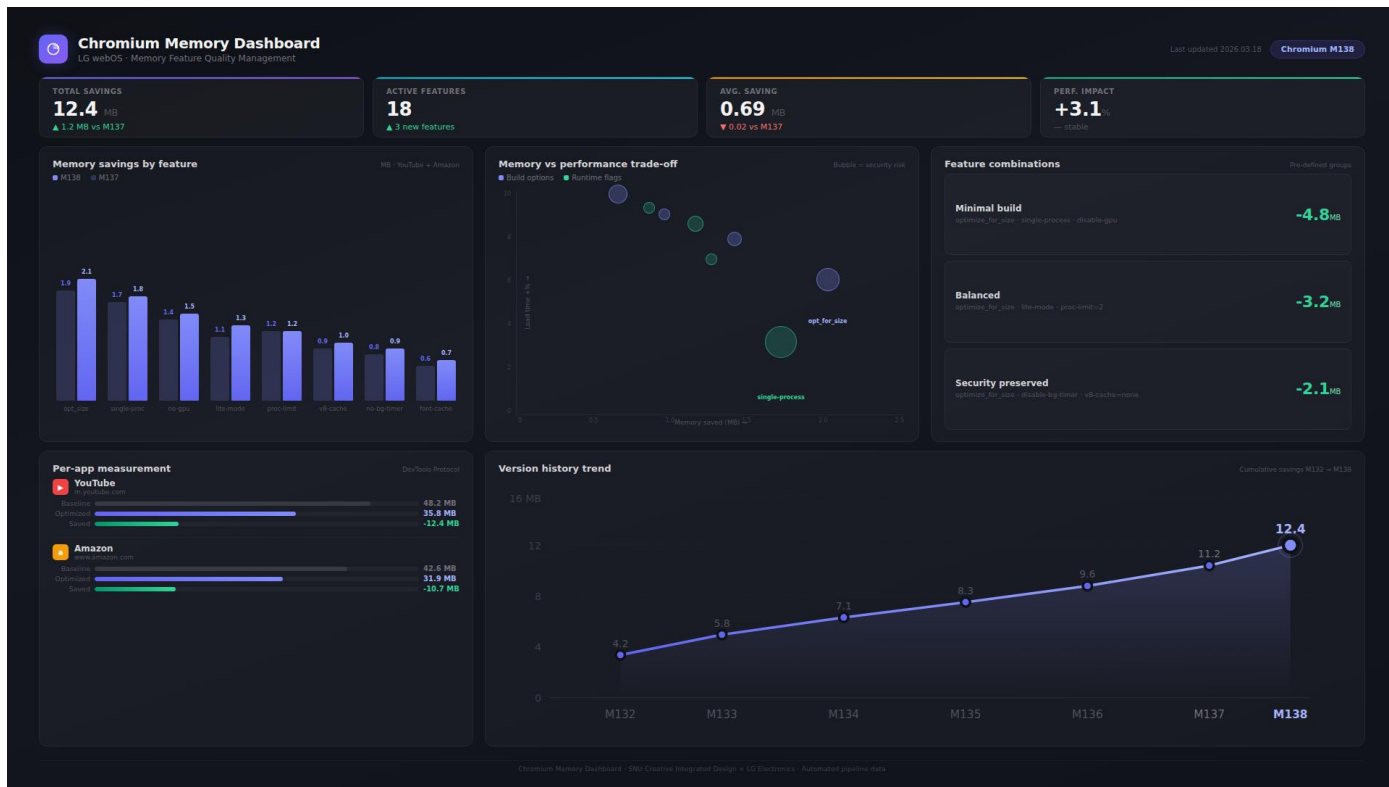
| 항목         | 세부 내용                                                    |
|------------|----------------------------------------------------------|
| 개발환경 Setup | Ubuntu 22.04 원격 접속 (SSH), Headless 모드 및 xvfb 가상 디스플레이 병행 |
| 메모리 측정     | CDP 연동 복합 지표 수집 / 대상 : 유튜브, 네이버 스포츠 등                    |
| 자동화 프레임워크  | 단일 Python 스크립트 기반 통합 제어 (Selenium 및 CDP 세션 연동)           |
| 기능 관리      | 적용 방식이 다른 런타임 플러그, 빌드 옵션, 코드 패치 자동 판별 및 실행 로직 분기         |
| 결과 출력      | 20+20초 시나리오(MAX값) 5회 반복 중간값 산출 / 스크린샷 등 중간 데이터 JSON 구조화  |

## 2. 메모리 기능 관리 대시보드

---

| 항목     | 세부 내용                                                |
|--------|------------------------------------------------------|
| 데이터 소스 | 자동화 파이프라인에서 생성된 JSON 파일                              |
| 주요 지표  | 단일 기능별 절감량, 버전·URL 별 트렌드 추이, 각 기능별 기술 요약 정보          |
| 시각화    | Chart.js를 활용한 바 차트, 트렌드 라인, 기능 조합 히트맵 등 직관적 렌더링      |
| 기능 조합  | 대시보드 내 새로운 조합을 동적으로 구성(On/Off)하고 시너지/Trade-off 수치 비교 |
| 구현 방식  | 백엔드(Flask) + 프론트엔드(Vanilla JS) 기반 자체 경량 웹앱 구현        |
| 사용자 경험 | 인터랙티브 분석 뷰 및 대시보드 내 파이프라인 원격 제어(Start/Stop) 콘솔 제공    |

## 2. 메모리 기능 관리 대시보드 - 샘플



# 3. 메모리 절감 목표 스펙

---

## **Ideal** 크로미움 오픈소스 생태계 기여 (Upstream)

작성한 메모리 절감 코드 패치를 크로미움 메인 커밋(Mainline)에 제안하여  
공식적으로 Accept 받는 것

## **Intermediate** 임베디드 맞춤형 최적화 조합 및 자체 패치 발굴

타겟 환경(메모리 1~2GB)에 최적화된 크로미움 빌드 조합 2~3개 도출  
불필요한 기능 제거나 효율화를 위한 의미 있는 코드 패치 0~1건 제작 및 검증

## **Baseline** 런타임 플래그 및 빌드 옵션 최적화

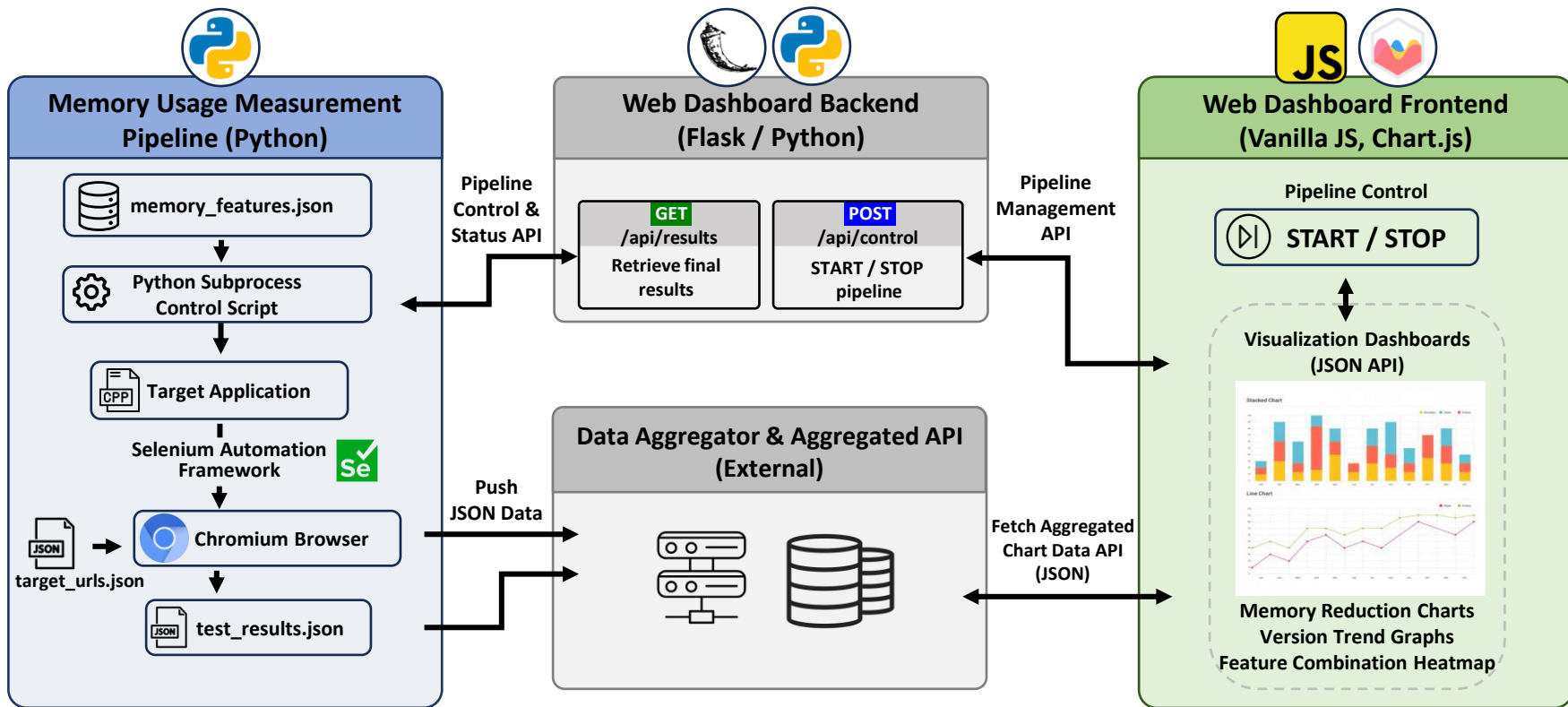
기존에 알려진 플래그와 옵션들을 조합하여,  
코드 수정 없이 적용 가능한 수준의 메모리 및 성능 최적화 달성

# System Architecture

---

시스템 아키텍처

# 시스템 아키텍처



# Work Plan

---

역할 분담 및 일정

# TaskList

---

## 전체 Task List

- **Task A:** 데이터 포맷(스키마) 표준화 | 측정 결과 및 설정값을 유연하게 관리하도록 포맷 구축
- **Task B-1:** CDP 연동 | 브라우저 내부 지표를 직접 측정하도록 변경
- **Task B-2:** 다각적 정량 지표 수집 | 메모리 외에 로딩 속도, CPU 점유율 등 추가 지표 확보
- **Task B-3:** 엄밀한 측정 시나리오 적용 | 20초 대기 및 20초 측정, 5회 반복 후 중간값 도출 로직 구현
- **Task B-4:** 원격 에이전트 구조 개편 | 안정성을 위해 실행 스크립트와 설정 데이터 분리
- **Task C-1:** 시너지 분석 및 차트 고도화 | 기능 조합 시 발생하는 성능 변화 및 효과를 비교하는 차트 구현
- **Task C-2:** 실시간 모니터링 위젯 도입 | 서버 상태, 작업 진행률 등을 확인하는 상태바 추가
- **Task C-3:** 직관적인 로그 시각화 | 빌드 및 실행 실패 시, 사용자가 원인을 쉽게 파악할 수 있는 UI 구성
- **Task D-1:** 기본 조합 검증 | 일반적인/유명한 옵션 조합의 파이프라인 정상 가동 확인
- **Task D-2:** 신규 최적화 조합 발굴 | 유의미한 메모리 절감 효과를 내는 새로운 플래그·옵션·패치 조합 탐색
- **Task D-3:** 중간발표 준비
- **Task E:** 향후 고도화 및 조합 탐색 | 새로운 조합·패치 발굴,
  - (중간발표까지 목표 달성 시) 대시보드 인프라 개선(대시보드에서 task 추가, 타겟 url 변경, 크로미움 버전 선택, 알림 기능 등), 측정값의 시계열 시각화, 파

# 역할분담 및 향후 일정

---

## 주요 마일스톤

- ~04/18 (일): 단기 목표
  - 공통: CDP 관련 스테디 및 대시보드 화면(UI) 기획안 확정 [B-1, B-2, C-1]
  - 노현제: 대시보드 프론트엔드 화면 구현 (실시간 모니터링 위젯, 차트 시각화 등 UX 개선) | [C-2, C-1, C-3] 중 일부
  - 손민규: 측정 엔진 및 백엔드 구축 (데이터 포맷 세팅, CDP 지표 수집 및 연동, 원격 파이프라인 통신 구조 개편) | [A, B-1, B-2, B-3, B-4] 중 일부
- ~05/01 (금): 중간발표 목표
  - 대시보드 필수 UI 구현 완료 및 파이프라인 정상 가동 확인, 대시보드 UX 향상
  - 유의미한 빌드 옵션이나 코드 패치 조합 최소 1개 발굴 및 인사이트 도출

---

\*일단은 위와 같이 프론트엔드(정: 노현제, 부: 손민규), 백엔드(정: 손민규, 부: 노현제)로 나눠서 진행하려고 계획하고 있고, 공통적으로 UI/UX 조사 및 의견공유, CDP 메서드 조사 및 메모리 절감 아이디어 연구, 추가 기능이나 파이프라인 고도화에 대한 아이디어 공유를 지속적으로 하면서 유동적으로 프로젝트를 진행하려고 합니다.

# 역할 분담 (수정 후)

## Part 1. 자동화 검증

노현제

- 크로미움 빌드
- 웹크롤링 로직 작성

손민규

- Selenium 자동화 스크립트 개발

한진우

- JSON 기반 API 설계 및 작성

## Part 2. 대시보드

노현제

- 웹 대시보드 UI 설계
- Javascript 프론트엔드 구현

손민규

- Flask 기반 백엔드 구현

한진우

- Javascript 시각화 컴포넌트 구현

## Part 3. 아이디어 발굴

노현제

- 크로미움 관련 프로젝트 코드 분석

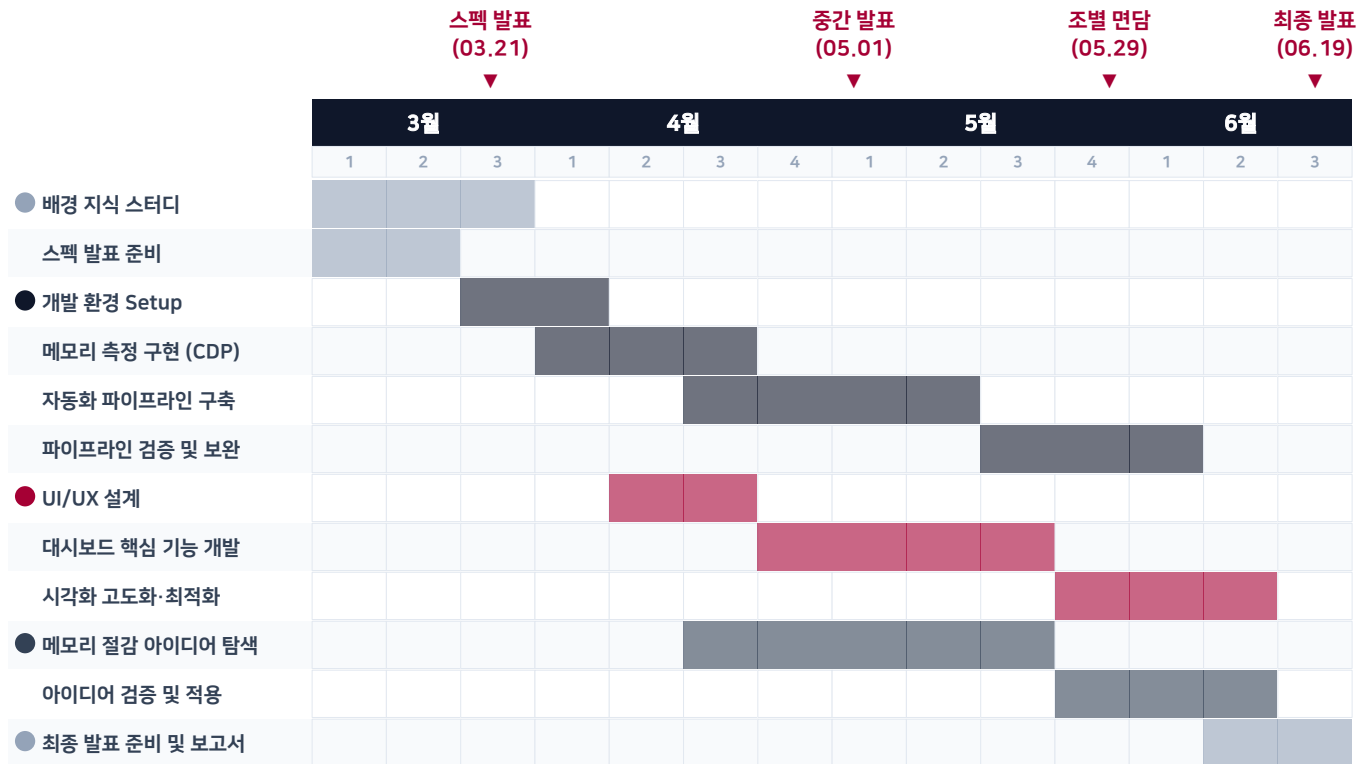
손민규

- 런타임 플래그, 빌드 옵션 조합  
시뮬레이션

한진우

- AI 프롬프트 엔지니어링

# 일정



# 감사합니다

---

노현제, 손민규, 한진우

Thanks to  
서울대학교 엄현상 교수님  
LG전자 송호진 책임연구원님 | 정경환 선임연구원님